Parcial Algoritmos y Programación II 7541 Curso Calvo (3º instancia) 29 de Junio de 2010

Padrón: **Apellido y Nombre:**

Mail:

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida

```
/* Datos:
 *   '@' es 64
 *   'A' es 65
 */
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 65;
  G = (char) H;
  A = new int;
  (*A) = H;
  F = A;
  C = F;
  (*A) = 67;
  cout << (*A) << (*C) << (*F) << endl;

  B = &C;
  C = new int;
  (**B) = H + 1;
  cout << (*A) << (*C) << (**B) << endl;

  E = (char*) C;
  D = (char*) (*B);
  cout << (*E) << (*D) << endl;


  (*E) = G;
  cout << (*E) << (*C) << endl;

  F = (int*) D;
  while ( (*C) > 0) {
      (*C)--;
      cout << (*C) << (*D) << (**B) << endl;
      (*F) = (*F) - (**B);
  }

  delete A;
  delete C;

  return 0;
}
```

2) Implementar para clase Cola las primitivas acolar y desacolar.

3) Implementar la clase AuditorDeConcursos a partir de las siguientes especificaciones:

```
class AuditorDeConcursos {
  public:
    /* post: busca en concursosAuditados aquellos Concursos para los cuales el promedio de los puntajes de los Participantes
     *       supera a puntajeMinimo.
     *       Devuelve una nueva Lista con todos los Concursos en esta condición. */
    Lista<Concurso*>* buscarConcursos(Lista<Concurso*>* concursosAuditados, int puntajeMinimo);
};
```

```
class Concurso {
  public:
    /* post: inicializa el consurso sin participantes
     *       asociados, con el nombre indicado.
     */
    Concurso(string nombre);
    /* post: destruye todos los participantes asociados.
     */
    ~Concurso();
    /* post: devuelve el nombre que identifica el consurso.
     */
    string getNombre();
    /* post: devuelve los participantes asociados (incluye al
     *       Participante ganador)
     */
    Lista<Participante*>* getParticipantes();
    /* post: devuelve el Participante ganador del concurso.
     */
    Participante* getGanador();
    /* post: indica qué participante ganó el concurso.
     */
    void setGanador(Participante* participante);
};
```

```
class Participante {

  public:
    /* post: inicializa el participantes con el nombre
indicado
     *       y 0 puntos.
     */
    Participante(string nombre);
    /* post: devuelve el nombre que los identifica
     */
    string getNombre();
    /* post: devuelve la cantidad de puntos del participante
     */
    int getPuntos();
    /* post: cambia por cantidad los puntos del participante
     */
    void setPuntos(int cantidad);
};
```

4) Diseñar la especificación e implementar los TDAs modelen una **Factura**, cumpliendo las siguientes características:
Una **factura** está compuesta por una secuencia de **Items**. Cada **Item** tiene descripción, cantidad, precio y monto (monto = precio x cantidad).
Se deben poder agregar **Items** en la **Factura** y obtenerlos a partir de su número.
El monto de la **Factura** es la suma de los montos de sus **Items**.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Mantener apagados celulares, i-pod, reproductores de mp3, etc. durante el examen
Duración del examen : 2 horas y 30 minutos